CLIPMIND AI

AI-Powered Video Summarization and Intelligent Key Moment Detection Platform

Milestone 1 Documentation

Project Initialization, Design Process and Core Setup

1. Introduction

ClipMind AI is a video-learning platform that helps users work with video content. Module 1, “Project Initialization, Design Process and Core Setup,” focuses on building the basic application and making sure its main parts work together.

This module covers the initial project setup, system design, database, main pages, user accounts, video uploads, upload history, processing status, and a basic FFmpeg workflow.

2. Objectives of Module 1

- Set up the frontend, backend, and database.
- Plan the main application pages and system structure.
- Allow users to register and log in securely.
- Provide dashboards based on user roles.
- Allow authorized users to upload videos and check their status.
- Save video details and show upload history.
- Run a basic video-processing operation using FFmpeg.
- Test the main features before moving to the next module.

3. Technology Stack

Component	Technology	Purpose
Frontend	React.js or Next.js	Build the pages and user interface.
Backend	Python FastAPI	Handle requests, authentication, uploads, and application logic.
Database	PostgreSQL or MongoDB	Store user details and video information.
Authentication	JWT	Protect pages and APIs that require login.
Password security	bcrypt or Argon2id	Store passwords as secure hashes.
Video processing	FFmpeg / FFprobe	Inspect and perform basic video operations.
API testing	Postman	Check whether backend APIs work correctly.

The team should agree on one frontend framework and one database before development starts.

4. Project Structure

```
ClipMind-AI/
├── frontend/       # User interface
├── backend/        # FastAPI application and APIs
├── database/       # Database scripts or notes
├── docs/           # Project diagrams and documentation
├── tests/          # Tests
├── .gitignore
└── README.md
```

The frontend contains the pages and reusable components. The backend contains the API routes, authentication, database operations, and video-processing logic. The docs folder holds diagrams and project notes. The README explains how to set up and run the application.

5. System Architecture

```
User
  ↓
Frontend (React.js / Next.js)
  ↓ API requests
FastAPI Backend
  ├── Authentication and user roles
  ├── Video upload and validation
  ├── Video status management
  ├── Database (user and video details)
  ├── File storage (actual video files)
  └── FFmpeg / FFprobe
```

The frontend displays information and sends requests to the backend. FastAPI checks the request, applies the required rules, and works with the database and video storage. FFmpeg is used for the initial video-processing step.

The video file itself should be stored in a file-storage location. The database stores details about the file, such as its name, location, uploader, upload time, and status.

6. Database Design

6.1 User

Field	Purpose
id	Unique user ID.
name	User's full name.
email	Email address used to log in.
password_hash	Hashed password; never store the plain password.
role	Content Creator, Learner, Educator, or Administrator.
created_at	Date and time the account was created.

6.2 Video

Field	Purpose
id	Unique video ID.
user_id	Identifies the user who uploaded the video.
filename	Video filename.
file_path	Location where the video is stored.
status	Current upload or processing status.
uploaded_at	Date and time of upload.
file_size	Size of the video file.

Each video should be linked to the user who uploaded it. The backend must check permissions before showing video details or allowing access to a stored file.

7. User Roles

Role	Main use in Module 1
Content Creator	Upload videos and check their upload history and status.
Learner	View learning content and results made available to them.
Educator	Access educator features defined by the project team.
Administrator	Manage permitted user and system functions.

The team should confirm the exact permissions for each role. The backend must enforce these permissions. Users must not be able to give themselves administrator access through the registration form.

8. Feature 1 — User Registration

8.1 Registration Form

- Full name
- Email address
- Password
- Confirm password
- Role selection, according to the application's account policy
- Register button and a link to Login

8.2 How It Works

```
User enters details
      ↓
Frontend checks required fields
      ↓
FastAPI validates the request
      ↓
Check whether the email is already registered
      ↓
Hash the password and save the user
      ↓
Show the registration result
```

The backend must validate the details even if the frontend checks them first. Passwords must be hashed before they are saved.

9. Feature 2 — Login and JWT Authentication

9.1 Login Form

- Email address
- Password
- Login button
- Link to the Registration page

9.2 Login Process

```
User enters credentials
      ↓
FastAPI verifies the email and password
```

```
      ↓
A signed JWT is created
      ↓
Frontend receives the authentication response
      ↓
User opens the appropriate dashboard
```

JWT (JSON Web Token) is used to identify an authenticated user when making protected API requests. The backend must check that the token is valid and has not expired. Keep signing secrets outside the source code, use HTTPS in production, and choose a secure way to store tokens.

10. Feature 3 — Role-Based Dashboard

After logging in, users should see a dashboard suited to their role. The layout can share common elements such as navigation, profile information, and logout.

Role	Dashboard content
Content Creator	Video upload, upload history, and processing status.
Learner	Learning content and results they are allowed to view.
Educator	Educator features agreed by the team.
Administrator	Authorized user and system management functions.

The dashboard should show useful messages when there are no videos, when data is loading, or when an operation fails.

11. Feature 4 — Video Upload and Validation

11.1 Upload Form

The upload page should let the user select a video, see its filename, and start the upload. It should display progress where possible and show a clear message when the upload succeeds or fails.

11.2 Upload Process

```
User selects a video
      ↓
Frontend checks basic requirements
      ↓
Backend checks login and upload permission
      ↓
Backend validates the file
      ↓
Save video and its details
      ↓
Return the video ID and status
```

11.3 Validation Checks

- Check that a file was selected.
- Set and enforce a maximum file size.
- Allow only supported video formats.
- Check that the file is a readable video; FFprobe can help inspect it.
- Use a safe internal filename and prevent unsafe file paths.
- Keep uploaded files out of publicly accessible folders.

The backend must repeat important validation checks rather than relying only on the browser.

12. Feature 5 — Video Storage and Upload History

The actual video file is stored in a designated storage location. The database stores the file's details, including who uploaded it, its filename, its storage location, and its current status.

History field	What it shows
Filename	Name shown to the user.
Upload date	When the video was uploaded.
File size	Size of the uploaded video.
Status	Current processing stage.
Action	A link or button to view authorized details.

Users should see only the videos they are permitted to access. If a user has not uploaded any videos, the page should show a helpful empty-state message.

13. Feature 6 — Processing Status and FFmpeg

13.1 Video Status

The application should show the current state of each video. Suggested status values are: UPLOADED, VALIDATING, QUEUED, PROCESSING, COMPLETED, and FAILED.

```
UPLOADED → VALIDATING → QUEUED → PROCESSING
                                    ├→ COMPLETED
                                    └→ FAILED
```

13.2 Basic FFmpeg Setup

FFmpeg will be used to establish the first video-processing operation. FFprobe, which comes with FFmpeg, can inspect the video format and streams. For example:

```
ffprobe -v error -show_format -show_streams -of json input.mp4
```

The team should choose the actual processing task required for this milestone. The backend should update the status when processing starts, finishes, or fails. Long-running work should not block an API request; a background task or worker can be used when needed.

Run media tools with safe file paths and suitable time and resource limits.

14. Frontend and Backend Integration

The frontend and FastAPI backend communicate through API requests. Keep API calls in a shared frontend service so the base URL and error handling can be managed in one place.

Method	Endpoint	Purpose
POST	/api/auth/register	Create an account.
POST	/api/auth/login	Log in.
GET	/api/auth/me	Get the current user's details.
GET	/api/dashboard	Get dashboard information.
POST	/api/videos/upload	Upload a video.
GET	/api/videos	Get permitted upload history.
GET	/api/videos/{video_id}	Get video details.
GET	/api/videos/{video_id}/status	Get the processing status.

These are suggested endpoints; the team should match them to the actual backend routes. Store the API base URL in configuration rather than repeating it throughout the frontend.

15. API Testing with Postman

Use Postman to test the backend before connecting every page in the frontend.

- Test successful registration and duplicate-email rejection.
- Test login with both correct and incorrect credentials.
- Check that protected APIs reject requests without a valid token.
- Upload a valid video and check its saved details.
- Test unsupported formats and files that exceed the size limit.
- Check upload history and processing status.
- Confirm that one user cannot access another user's private video.

Record the expected result and actual result for each important test.

17. Expected Outcome

By the end of Module 1, ClipMind AI should have a working basic application. Users should be able to register, log in, open the dashboard for their role, upload supported videos, view their upload history, and check processing status.

The backend should store video details, protect user data, and run a basic FFmpeg-based operation. This foundation will make it easier to add the platform's more advanced video-learning features in later modules.

